

Summary

This video provides the foundational principles of the Theory of Computation (TOC), focusing on the mathematical representation of formal languages through automata. The objective is to explain the hierarchy of computational models—Finite Automata (FA), Pushdown Automata (PDA), Linear Bounded Automata (LBA), and Turing Machines (TM)—and compare their expressive power.

Key Takeaways:

- **Expressive Power Hierarchy:** Turing Machines > Linear Bounded Automata > Pushdown Automata > Finite Automata.
- **Determinism vs Non-determinism:** DFA and NFA have equal expressive power. However, NPDA is strictly more powerful than DPDA.
- **Mathematical Foundations:** Computation begins with symbols, which form alphabets (Σ), which generate strings. The number of strings of length n over Σ is $|\Sigma|^n$.

1. Core Definitions and Automata Modeling

Automata are mathematical models designed to accept all strings belonging to a language and reject those that do not.

1.1 The Modeling Process

Consider the language:

$$L = \{ a^n \mid n \geq 1 \}$$

Language Structure:

a, aa, aaa, aaaa, ...

Model Construction:

- Start at initial state A.
- On reading first 'a', transition to final state B.
- Add self-loop on B for symbol 'a' to accept longer strings.

This demonstrates the principle of minimal modeling: define the smallest accepting string, then generalize via loops.

2. Classification and Expressive Power of Automata

Expressive Power (E) is defined as the number of language classes a model can recognize.

2.1 Chomsky Hierarchy and Automata Types

Automata Type	Language Accepted	Language Type	Expressive Power (E)
Finite Automata (FA)	Regular Languages	Type 3	1
Pushdown Automata (PDA)	Context-Free Languages (CFL)	Type 2	2
Linear Bounded Automata	Context-Sensitive Languages (CSL)	Type 1	3
Turing Machine (TM)	Recursively Enumerable Languages	Type 0	4

2.2 Relative Power Comparisons

A model m_1 is more powerful than m_2 if:

$$E(m_1) > E(m_2)$$

Key Comparisons:

- PDA > FA (every Regular language is a CFL).
 - LBA > PDA > FA.
 - TM is the most powerful model and can accept REL, CSL, CFL, and Regular languages.
- ### 3. Deterministic vs Non-deterministic Systems

A deterministic system is complete; a non-deterministic system may be incomplete in transition definition.

3.1 NFA vs DFA

Feature	NFA	DFA
System Type	Incomplete	Complete
Invalid Input	Simply not accepted	Explicitly rejected
Expressive Power	Equal to DFA	Equal to NFA
Conversion	NFA $\rightarrow$ DFA possible	DFA is special case of NFA

Conclusion:

DFA and NFA have equal expressive power.

3.2 Power Variations in Higher Automata

Pushdown Automata:

- NPDA is strictly more powerful than DPDA.
- Example: Language WW^R can be accepted by NPDA but not DPDA.

Linear Bounded Automata:

- Relationship between DLBA and NDLBA remains an open research question.

Turing Machines:

- Deterministic and Non-deterministic TMs have equal expressive power.
- Single-tape and multi-tape TMs are equivalent in power.

4. Fundamental Components of Computation

4.1 Symbols and Alphabets

Symbol:

An indivisible atomic entity (e.g., a, b, 0, 1).

Alphabet (Σ):

A non-empty finite set of symbols.

Types of Alphabets:

- Unary: $\{0\}$
- Binary: $\{0,1\}$ or $\{a,b\}$
- Ternary: $\{0,1,2\}$

Constraint:

Alphabet must be finite; otherwise, formal learning and computation become undefined.

4.2 Strings and Length

String (w):

A finite sequence of symbols from Σ .

Length $|w|$:

Total number of symbols in w .

Empty String (ϵ):

- Length = 0.
 - It is a string, not a symbol.
 - Cannot be an element of Σ .
5. Mathematical Calculations for Strings

5.1 Strings of Specific Length

Number of strings of length n over Σ :

$$\text{Total Strings} = |\Sigma|^n$$

Examples:

Binary Alphabet ($|\Sigma| = 2$):

$$\text{Total strings of length } n = 2^n$$

Ternary Alphabet ($|\Sigma| = 3$):

$$\text{Total strings of length } n = 3^n$$

5.2 Total Strings Up to Length n

Total strings from length 0 to n :

$$\text{Total} = (|\Sigma|^{n+1} - 1) / (|\Sigma| - 1)$$

Binary Alphabet ($|\Sigma| = 2$):

$$\begin{aligned} \text{Total} &= (2^{n+1} - 1) / (2 - 1) \\ &= 2^{n+1} - 1 \end{aligned}$$

Example:

$$\text{Up to length } 5 \rightarrow 2^6 - 1 = 63$$

Ternary Alphabet ($|\Sigma| = 3$):

$$\begin{aligned} \text{Total} &= (3^{n+1} - 1) / (3 - 1) \\ &= (3^{n+1} - 1) / 2 \end{aligned}$$

Example:

$$\begin{aligned} \text{Up to length } 5 &\rightarrow (3^6 - 1) / 2 \\ &= (729 - 1) / 2 \\ &= 364 \end{aligned}$$

6. Significant Insights and Pedagogical Observations

On Quality of Documentation:

Notes should be constructed with long-term clarity and durability in mind, serving not only immediate preparation but future academic reference.

On Determinism in Practical Systems:

Real-world systems such as ATMs behave like DFAs: they explicitly handle both valid and invalid inputs.

On NPDA vs DPDA:

Conversion from NPDA to DPDA is not possible in general because their expressive powers differ.

On Turing Machine Equivalence:

Deterministic and Non-deterministic Turing Machines, as well as single-tape and multi-tape versions, are equivalent in expressive power.

